

PROJE 02 / 20 · ÇALIŞAN TEMEL SÜRÜM

Canlı Piyasa İzleme Paneli

Simüle fiyat akışını, favorileri ve üst eşik alarmlarını bir arayüzde izle.

Simüle fiyatlar · SSE akışı

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 02 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

Canlı Piyasa İzleme Paneli

Simüle fiyat akışını, favorileri ve üst eşik alarmlarını bir arayüzde izle.

Simüle fiyatlar · SSE akışı

Canlı simülâtör bağıliSnapshot yenile

BTC-DEMO

59990.46

Sıra 1 · 12:33:23 PM

Favoriye ekle

ETH-DEMO

3009.25

Sıra 2 · 12:33:23 PM

Favoriye ekle

VUE-DEMO

101.90

Sıra 3 · 12:33:23 PM

Favoriye ekle

Son 100 fiyat noktası

Üst eşik alarmı

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

3 varlık için iki saniyede bir fiyat · Server-Sent Events yayını · 100 noktalı SVG çizgi grafik · Kullanıcıya özel favoriler · Üst eşik geçişinde tek alarm

Bu sürümde olmayanlar

Gerçek borsa/kripto API'si yok. Tek yönlü SSE kullanılır; WebSocket değildir. Her sunucu yeniden başlatıldığında fiyat simülâtörü devam eder; gerçek piyasa geçmişi veya mum grafiği yok.

DENİZ YARDIMCI · FULL-STACK LAB · 23.09.2026

1

02 / Backend ve veri modeli

Backend fiyat noktalarına artan seq verir. Arayüz eski sequence değerini işlemez. Alarm önceki fiyat eşik altında/eşik ve yeni fiyat eşik üstünde olduğunda tek kez tetiklenir, armed=0 olur. ticks tablosu son 1500 kayıtlı sınırlandırılır. SSE oturum gerektirir; kişisel alarm yalnız sahibine gönderilir.

Gerçek SQL şeması

```
CREATE TABLE ticks(seq INTEGER PRIMARY KEY AUTOINCREMENT,symbol TEXT,price REAL,at TEXT);
```

```
CREATE TABLE watch(owner TEXT REFERENCES users(id),symbol TEXT,PRIMARY KEY(owner,symbol));
```

```
CREATE TABLE alerts(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),symbol TEXT,threshold REAL,armed INTEGER DEFAULT 1,triggered_at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/market	Gövde yok	200
GET /api/prices/:symbol	Gövde yok	200
PUT /api/watch/:symbol	Gövde yok	200
DELETE /api/watch/:symbol	Gövde yok	200
POST /api/alerts	JSON: symbol, threshold	201
DELETE /api/alerts/:id	Gövde yok	200

Somut istek örneği

```
POST /api/alerts
{"symbol": "BTC-DEMO", "threshold": 61000}
201 {"id": "<alert-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error": "açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Aynı favori iki PUT isteğinde tek satır olmalı.
- Üç sembolün fiyatı ve artan sıra numarası görünmeli.
- Geçersiz sembolle alarm oluşturma 422 dönmeli.
- İkinci kullanıcının favorileri ayrı kalmalı.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const market=await get('/market');assert.equal(market.prices.length,3);
await call('/api/watch/BTC-DEMO','PUT',{},200);await call('/api/watch/BTC-DEMO','PUT',{},200);
assert.equal((await get('/market')).watch.length,1);
assert.equal((await call('/api/market','GET',undefined,200,other)).watch.length,0);
await post('/alerts',{symbol:'INVALID',threshold:10},422);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Fiyat tablosunu ve favori işlemini incele. İkinci tarayıcı oturumunda favorilerin ayrıldığını göster; snapshot isteğinin sonucunu kaydet.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Bağlantı geri geldiğinde otomatik snapshot al
- 2. Veri eskiliği rozeti ekle
- 3. Sağlayıcı adaptörü ve kota yönetimi ekle
- 4. Yatay ölçek için ortak yayın kanalı tasarla

Anlamli commit sırası

```
git add .
git commit -m "chore: add market-watch baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

SSE neden burada yeterli? Kopuk bağlantı sırasında kaçan fiyat noktaları ve alarmlar nasıl tamamlanır?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.